

Group Project C: Using matrix and tensor decompositions to explore and make predictions on publication data

Objective

The objective of this exercise is to use matrix and tensor factorizations to **(a)** explore a dataset and **(b)** attempt to make publication predictions for the following year. The data comes from DBLP¹, a computer science publication database.

Detailed steps

The dataset can be downloaded [here](#). Four variables are stored in the file: **X**, **Y**, **Author_names** and **Conf_names**. **X** is a three-way array reflecting publications of authors between for the years 1991-2004, and is of size 471 (*authors*) x 366 (*conferences*) x 14 (*years*). The goal is to predict publications for the year 2005. To quantify the accuracy of the methods the ground truth for 2005 is supplied in **Y**. **Author_names** (471 total) and **Conf_names** (366 total) contain strings of the author and conference names in **X** and **Y**.

Data exploration

We begin with some data exploration. Start to get acquainted with the data by answering the following questions:

1. Which are the top five authors in terms of number of publications?
2. Which are the top ten conferences in terms of publications?
3. Which pairs of (authors, conferences) appear at the highest frequency? In other words, if we could frame the data in tuples of the form (authorX, conferenceY, total_no_of_publications_of_authorX_to_conferenceY), which are the top five tuples if we sort them according to the last field?
4. Is there some conferences not happening every year?
5. What is the percentage of zeros and non-zeros in the dataset?

¹ <https://dblp.org/>

Using the SVD

Begin by log-transforming the data, i.e. each $\mathbf{X}_{ijk} = \log(\mathbf{X}_{ijk}) + 1$, where i, j and k and $\mathbf{X}_{ijk} \neq 0$.

SVD can only be applied to matrix data, therefore implement and experiment with the following approaches of turning the three-way tensor to a matrix:

1. Sum the publications of each author across the years into a single matrix $\mathbf{Z}$.
2. Sum the publications of each author across the years into a single matrix $\mathbf{Z}$ but weighting them according to

$$Z(i, j) = \sum_{t=1}^T (1 - \theta)^{T-t} X(i, j, t)$$

Compute the best rank-K approximation of $\mathbf{Z}$ for different values of $K = \{2, 10, 20, 50, 100, 300\}$. Entries of each rank-K estimate $\mathbf{Z}_k$ can be used to predict the values in 2005. In this context, we only want to predict if there is a publication in 2005 from each author to each conference, therefore you should replace all non-zero entries with 1 in $\mathbf{Y}$, so that $\mathbf{Y}$ contains the binary true labels. Then, answer the following:

1. For both approaches, measure the reconstruction error as K increases, what do you observe?
2. Give an intuition on the formula of approach 2 – what does it assume?
3. *Predict the future!* When attempting to predict the binary $\mathbf{Y}$ from each $\mathbf{Z}_k$, plot ROC curves and choose the best performing K and give, in each case, the *AUC*.
4. Compare different combinations of K and $\theta = \{0.01, 0.1, 0.25, 0.5, 0.75, 1.0\}$ in the above formula and find the best-performing combination(s). What would you use to predict?

Tensor factorization

Now you are going to go through the actual research pipeline used in reality when analyzing data with tensor decompositions, namely the CANDECOMP/PARAFAC (CP) decomposition. Please note that all these names are used interchangeably both in literature and in library documentation, since the same method was proposed in multiple domains approximately at the same time by different people. Follow the steps below, presenting your results at each one:

1. Fit CP models with rank=2. Run at least ten-twenty random initializations, and compare the resulting decompositions in terms of Factor Match Score (FMS). To

compute CP, you may use TensorLy² and to compute FMS you may use the `factor_match_score`³ from TLViz. What do you observe? What is the fit? Include in your results the execution time and total number of iterations taken to converge.

2. Repeat the above process for $R=\{1, 4, 6, 8, 10, 12, 15, 20, 25, 50\}$ and compare the results in each case. What do you observe?
3. Choose a specific rank that you find suitable and plot a component, i.e. a specific column of each factor matrix. Using **Author_names** and **Conf_names**, interpret the component. Look at the raw (unprocessed data). Does what you found make sense?
4. For the different R values, choose the run with the lowest total loss as a representative. Reconstruct the complete tensor from the factors (Hint: use `cp_to_tensor`⁴) and attempt to predict **Y** by averaging the last three time-points of the tensor, i.e. $\mathbf{X}_{\text{reconstructed}}(:, :, -3:)$. What do you observe?

Deliverables

You are to deliver a report with plots and discussion of your results. You are encouraged to also provide a jupyter notebook reproducing your results.

Acknowledgements

We would like to thank Evrim Acar and Tamara Kolda for allowing us access to the data. Further, we would like to thank Evrim Acar for the inspiration for the assignment.

References

[1] Dunlavy, Daniel M., Tamara G. Kolda, and Evrim Acar. "Temporal link prediction using matrix and tensor factorizations." ACM Transactions on Knowledge Discovery from Data (TKDD) 5.2 (2011): 1-27.

[2] Jean Kossaifi, Yannis Panagakis, Anima Anandkumar and Maja Pantic, TensorLy: Tensor Learning in Python, Journal of Machine Learning Research, Year: 2019, Volume: 20, Issue: 26, Pages: 1–6.

²

<https://tensorly.org/dev/modules/generated/tensorly.decomposition.parafac.html#tensorly.decomposition.parafac>

³ https://tensorly.org/viz/stable/api/factor_tools.html#tlviz.factor_tools.factor_match_score

⁴ https://tensorly.org/dev/modules/generated/tensorly.cp_tensor.cp_to_tensor.html

[3] Roald, Marie, and Yngve Mardal Moe. "TLViz: Visualising and analysing tensor decomposition models with Python." *Journal of Open Source Software* 7.79 (2022): 4754.

Grading / Assessment

(10%) Data exploration

(40%) Using the SVD

(50%) Tensor factorization